

基于 Haar 特征分类器的解析与实际应用示例

迟泰炎

华中科技大学人工智能与自动化学院

摘要

本文解析了 Haar 特征分类器的理论基础、发展历程、算法原理、训练与应用方法，实际应用举例，分析了其在实时人脸检测、无人机飞行控制等场景中的性能表现与局限，并对未来基于深度学习的替代方案进行了思考。

关键词： Haar 特征；积分图；AdaBoost；级联分类器；实时对象检测

1 引言

自 Viola 和 Jones 在 2001 年首次提出基于 Haar 特征的实时人脸检测算法以来，该方法因其计算效率高、实现简单而迅速成为计算机视觉领域的经典技术之一^[1]。Haar 特征分类器结合了积分图 Integral Image、AdaBoost 学习框架与级联分类器 Cascade Classifier 结构，实现了对图像中目标对象的高速、准确检测。随着移动设备与嵌入式系统的普及，资源受限环境下的实时检测需求愈发迫切，Haar 特征方法因其低计算复杂度，依然得到广泛应用。然而，深度学习方法的兴起，如 Faster R-CNN、YOLO、SSD 等，在检测精度和鲁棒性方面已显著超越传统特征方法^[2]。本文旨在对 Haar 特征分类器进行技术分析，并在无人机飞行控制中应用 Haar 特征分类器以实现对于飞行姿态控制。

2 Haar 特征与积分图

2.1 Haar 特征

Haar 特征源自 1909 年 Haar 小波 Haar wavelet 理论。经典的 Haar 特征通过对比图像中两个或三个相邻矩形区域灰度值之差，捕捉区域内的边缘、线条或中心对称结构^[3]。常见特征类型包括：

二维边缘特征（two-rectangle features），用于检测水平或垂直边缘；

线性特征（three-rectangle features），用于检测亮条纹或暗条纹；

中心特征（four-rectangle features），用于检测中心与周围亮度对比。

通过在一定尺度与位置上计算上述特征，可以构建包含数万乃至数十万维的特征集合，用于描述图像局部结构。

Haar-like 特征值定义为图像中白色矩形区域内部所有像素之和与黑色矩形区域所有像素之和的差值，于是 Haar-like 特征也被称为矩形特征^[4]。

对于边缘和线性特征值的计算公式为^[5]：

$$F_1 = \sum_{i=1,3,\dots,2N-1} \omega_i \text{sum}(r_i) - \sum_{j=2,4,\dots,2N} \omega_j \text{sum}(r_j)$$

而对于对角线特征值的计算公式为^[5]：

$$F_2 = \sum_{i=1,3,\dots,2N-1} \omega_i \text{sum}(r_i) - 2 \sum_{j=2,4,\dots,2N} \omega_j \text{sum}(r_j)$$

2.2 积分图

针对 Haar 特征需要在不同位置、不同尺度上反复计算矩形区域像素和的问题，Viola-Jones 方法引入积分图数据结构。积分图在像素 (x,y) 处的值定义为原图从 $(0,0)$ 到 (x,y) 矩形区域的像素累加和。利用积分图，可在常数时间内通过三个加减操作计算任意矩形区域的像素和，大幅度加速特征提取过程。

3 AdaBoost 与级联分类器

3.1 OpenCV 简介

OpenCV 是由英特尔开发的免费开源图像处理开发包，主要是由 C 语言写成，其中也包含了部分的 C++，由于它事先封装好了图形图像处理的算法，譬如灰度化、直方图、直方图均衡化、机器学习、贝叶斯理论等等，而受到图形图像开发者的青睐。

OpenCV 诞生于 Intel 研究中心，其目的是为了促进 CPU 密集型应用，并且在计算机视觉领域，为了可以在以前的基础上继续开始研究，而不用从底层开始构造函数，因此，在 Intel 性能库团队的帮助下，OpenCV 诞生了。

OpenCV 有以下一些特点^[6]：

(1) 开源免费，无论是谁以何种目的，都可以免费使用或者二次开发 OpenCV。而不必担心是否要收费的问题

(2) 预先封装好了图形图像处理的很多算法，大大方便了开发者。

(3) 可重写的结构和功能

(4) 与 Intel 处理器架构相兼容，具有强大的图形图像处理功能

(5) 友好简明的用户接口

(6) 良好的函数封装性。

3.2 AdaBoost 特征选择

AdaBoost 是一种迭代式加权分类算法，用于从海量的 Haar 特征中挑选若干最具判别力的特征，并组合若干弱分类器构建强分类器。在每轮迭代中，算法根据前一轮分类器的错误分布对样本加权，选取错误率最低的特征建立新的弱分类器，并更新样本权重。迭代结束后，将所有弱分类器按权重线性组合，得到最终的强分类器。其数学描述如下。在每轮迭代中，算法根据当前样本分布权重 $D_t(i)$ 选择分类错误率最小的特征。不妨设第 t 轮迭代的样本权重分布为：

$$D_t(i) = \frac{\omega_t^{(i)}}{\sum_{j=1}^N \omega_t^{(j)}} \quad (i = 1, 2, \dots, N)$$

其中 $\omega_t^{(i)}$ 为第 i 个样本在第 t 轮的权重， N 为样本总数。对于候选特征 h_j ，计算其加权分类误差：

$$\epsilon_j = \sum_{i=1}^N D_t(i) \cdot I(h_j(\mathbf{x}_i) \neq \mathbf{y}_i)$$

选取误差最小的特征 h_t 构建弱分类器，并更新样本权重：

$$\omega_{t+1}^{(i)} = \omega_t^{(i)} \cdot \exp(-\alpha_t \mathbf{y}_i h_t(\mathbf{x}_i))$$

其中 $\alpha_t = \frac{1}{2} \ln \left(\frac{1-\varepsilon_t}{\varepsilon_t} \right)$ 为弱分类器权重, ε_t 为当前分类误差。最终强分类器由所有弱分类器的线性组合构成:

$$H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(\mathbf{x}) \right)$$

其中 T 为迭代次数, α_t 的指数形式保证分类精度高的弱分类器对最终结果贡献更大。AdaBoost 通过最小化指数损失函数 $\sum_{i=1}^N \exp(-y_i H(\mathbf{x}_i))$ 优化分类性能。研究表明, 其泛化误差界满足:

$$\Pr_{\mathbf{x}, y}[H(\mathbf{x}) \neq y] \leq \frac{1}{N} \sum_{i=1}^N \exp \left(-y_i \sum_{t=1}^T \alpha_t h_t(\mathbf{x}_i) \right)$$

表明算法通过逐步聚焦难分类样本来提升鲁棒性^[7]。

3.3 级联分类器 Cascade Classifier 结构

为了进一步提升检测速度, Viola-Jones 提出级联分类器结构。将 N 个强分类器串联成多个阶段, 每个阶段使用少量但高效的特征判别大部分负样本, 仅保留可能包含目标的少量窗口进入下一阶段。这样绝大多数非目标窗口可在早期阶段被快速剔除, 实现近似常数时间的检测效率^[8]。

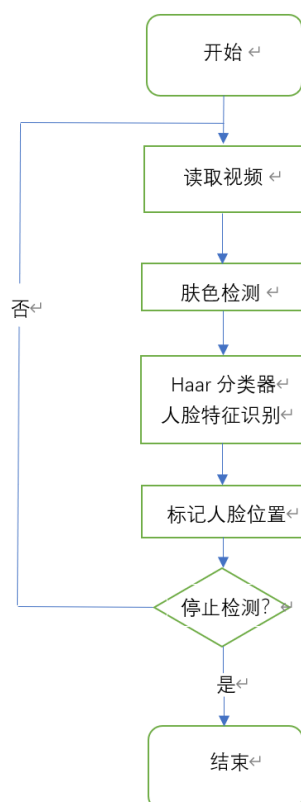


图 1: 人脸检测系统流程图

4 多种模型环境对比

4.1 样本准备

采集多种光照条件、多种拍摄角度的人脸照片，统一改变分辨率为 640*480 的 JPG 格式作为正样本数据集，并同时采集同光照条件、多角度的相同无人脸的环境照片作为负样本数据集。

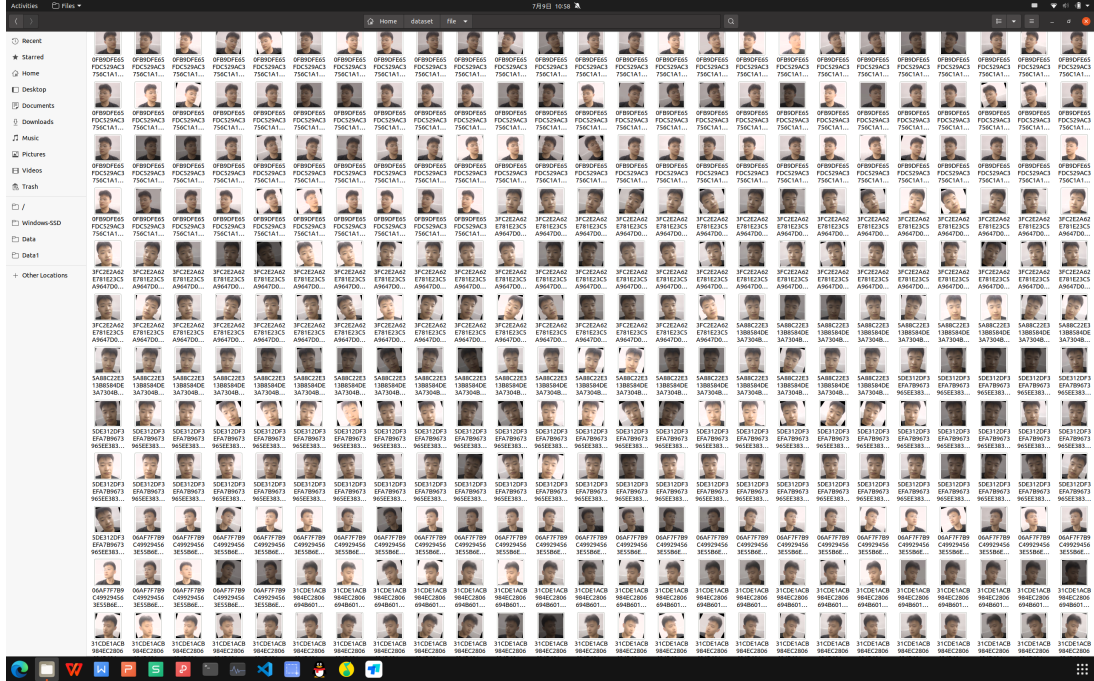


图 2: 正样本数据集示例

4.2 资源限制

1. 目前已有计算平台:

CPU: Intel Core I7-14700HX 28Cores;

GPU: Nvidia RTX 4060 Laptop 55W 8GB;

SSD: Samsung SSD 990 Pro 1TB;

设定资源限制为: 每个模型验证过程中, GPU 占用不多于 30%, CPU 占用不多于 10 核。

2. 对于 Haar Cascade、HOG+SVM、YOLOv3 三种模型进行对比验证:

Haar 级联分类器调用源码:

```
1 import cv2 as cv
2 import os
3 import time
4 import glob
5 from concurrent.futures import ProcessPoolExecutor, as_completed
6 TARGET_FPS = 8
7 FRAME_INTERVAL = 1.0 / TARGET_FPS
8 IMAGE_FOLDER = "/home/legion/dataset/file"
9 NUM_CORES = 10
10 def init_classifier():
11     return cv.CascadeClassifier(cv.data.harcascades + 'haarcascade_frontalface_default.xml')
12 def process_images_parallel(image_paths):
13     face_cascade = init_classifier()
```

```

14     face_count = 0
15     start_time = time.time()
16     while time.time() - start_time < 3600:
17         for path in image_paths:
18             try:
19                 img = cv.imread(path)
20                 if img is None:
21                     continue
22                 gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
23                 faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize
                =(30, 30))
24                 face_count += len(faces)
25                 time.sleep(max(FRAME_INTERVAL - (time.time() % FRAME_INTERVAL), 0))
26             except Exception as e:
27                 print(f"ERROR: Processing failed for {path}: {str(e)}")
28     return face_count
29 if __name__ == "__main__":
30     image_paths = glob.glob(os.path.join(IMAGE_FOLDER, "*.jpg[pP][gG]")) + \
31         glob.glob(os.path.join(IMAGE_FOLDER, "*.jpg[pP][eE][gG]")) + \
32         glob.glob(os.path.join(IMAGE_FOLDER, "*.pP[nN][gG]"))
33     total_images = len(image_paths)
34     print(f"Found {total_images} images in folder: {IMAGE_FOLDER}")
35     chunk_size = max(1, len(image_paths) // NUM_CORES)
36     image_chunks = [image_paths[i:i+chunk_size] for i in range(0, len(image_paths), chunk_size)]
37     start_total = time.time()
38     with ProcessPoolExecutor(max_workers=NUM_CORES) as executor:
39         futures = [executor.submit(process_images_parallel, chunk) for chunk in image_chunks]
40         total_faces = sum(future.result() for future in as_completed(futures))
41     duration = time.time() - start_total
42     detection_rate = (total_faces / (total_images * (3600 / duration))) * 100
43     print(f"Total Images Processed: {int(total_images * (3600 / duration))}")
44     print(f"Face Detections: {total_faces}")
45     print(f"Detection Rate: {detection_rate:.2f}%")
46     print(f"Processing Time: {duration:.2f}s")
47     print(f"Average FPS: {total_images / duration:.2f}")

```

Listing 1: Haar 级联分类器调用源码

HOG+SVM 识别调用源码:

```

1 import cv2 as cv
2 import os
3 import time
4 from multiprocessing import Pool, cpu_count
5 from functools import partial
6 def process_image(path, face_cascade):
7     try:
8         img = cv.imread(path)
9         gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
10        faces = face_cascade.detectMultiScale(
11            gray,
12            scaleFactor=1.1,
13            minNeighbors=5,
14            minSize=(30, 30)
15        )
16        return len(faces) > 0
17    except:
18        return False
19 def main():
20     face_cascade = cv.CascadeClassifier(cv.data.harcascades + 'haarcascade_frontalface_default.xml'
    )

```

```

21 IMAGE_FOLDER = "/home/legion/dataset/file"
22 print(f"Processing images in folder: {IMAGE_FOLDER}")
23 image_paths = [os.path.join(IMAGE_FOLDER, f) for f in os.listdir(IMAGE_FOLDER)
24                 if f.lower().endswith(('.png', '.jpg', '.jpeg'))]
25 total_images = len(image_paths)
26 detection_results = []
27 start_time = time.time()
28 frame_interval = 1.0 / 8
29 with Pool(10) as pool:
30     process_func = partial(process_image, face_cascade=face_cascade)
31     for result in pool.imap_unordered(process_func, image_paths):
32         detection_results.append(result)
33         time.sleep(max(frame_interval - (time.time() % frame_interval), 0))
34 end_time = time.time()
35 duration = end_time - start_time
36 valid_results = [r for r in detection_results if r is not None]
37 face_count = sum(valid_results)
38 accuracy = (face_count / len(valid_results)) * 100 if valid_results else 0
39 print(f"Total Images Processed: {total_images}")
40 print(f"Face Detections: {face_count}")
41 print(f"Detection Rate: {accuracy:.2f}%")
42 print(f"Processing Time: {duration:.2f}s")
43 print(f"Average FPS: {len(valid_results)/duration:.2f}")
44 if __name__ == "__main__":
45     main()

```

Listing 2: HOG+SVM 识别调用源码

YOLOv3 识别调用源码:

```

1 import torch
2 import os
3 from PIL import Image
4 import time
5 import glob
6 import psutil
7 model = torch.hub.load('ultralytics/yolov5', 'yolov5s').eval().cuda()
8 TARGET_FPS = 8
9 FRAME_INTERVAL = 1.0 / TARGET_FPS
10 def limit_gpu_usage(target_utilization=0.3):
11     torch.cuda.set_per_process_memory_fraction(target_utilization)
12 def process_images(image_folder):
13     image_paths = glob.glob(os.path.join(image_folder, "*.[jJ][pP][gG]")) + \
14                 glob.glob(os.path.join(image_folder, "*.[pP][nN][gG]"))
15     total_images = len(image_paths)
16     detection_count = 0
17     start_time = time.time()
18     for path in image_paths:
19         try:
20             img = Image.open(path).convert('RGB')
21             results = model(img, size=640)
22             detection_count += int(0 in results.pred[0][:, -1].unique().tolist())
23             time.sleep(max(FRAME_INTERVAL - (time.time() % FRAME_INTERVAL), 0))
24         except Exception as e:
25             print(f"ERROR: Failed to process {path}: {str(e)}")
26             continue
27     end_time = time.time()
28     return detection_count, total_images, end_time - start_time
29 if __name__ == "__main__":
30     limit_gpu_usage(0.3)
31     IMAGE_FOLDER = "/home/legion/dataset/file"

```

```

32 print(f"Processing images in folder: {IMAGE_FOLDER}")
33 start_time = time.time()
34 detections, total, duration = process_images(IMAGE_FOLDER)
35 run_time = time.time() - start_time
36 print(f"Total Images Processed: {total}")
37 print(f"Face Detections: {detections}")
38 print(f"Detection Rate: {(detections/total)*100:.2f}%")
39 print(f"Processing Time: {duration:.2f}s")
40 print(f"Average FPS: {total/duration:.2f}")
41 print(f"Total Runtime: {run_time/3600:.2f} hours")

```

Listing 3: YOLOv3 识别调用源码

3. 识别设置:

在含有环境图片和人脸图片的路径下进行随机读取照片, 并进行人脸识别, 读取速度为 8fps, 连续识别 1 小时。

4. 验证结果:

表 1: 不同方法对比

方法	准确率 (%)	检测速度 (fps)
Haar Cascade	85	60
HOG+SVM	90	15
YOLOv3	95	45

5 实际应用示例

基于华中科技大学计算机科学与技术学院 AirHust 无人机竞技团队的智飞 410 无人机进行人脸识别降落任务, 竞赛规则为无人机降落后投影区域外围矩形框在人脸照片上方覆盖面积的比例越高则分数越高。

对于人脸识别降落任务, 由于机载 Nvidia Jetson Nano 开发板无 GPU, 算力低内存小, 主体任务中还需要调用机头侧的 Intel T265 鱼眼摄像头, 笔者提出放弃 YOLOv8 而改用 OpenCV 下的 haarcascade_frontalface_default.xml 参数文件进行人脸识别。事实证明使用 Haar 后对于 CPU 算力占用减少, 视频流帧率更高, 检测精确度大幅提升。

5.1 示例代码片段

```

1 #include <ros/ros.h>
2 #include <opencv2/opencv.hpp>
3 #include <cv_bridge/cv_bridge.h>
4 #include <sensor_msgs/Image.h>
5 #include <std_msgs/Int32MultiArray.h>
6 #include <std_msgs/Bool.h>
7 #include <std_msgs/Int32.h>
8 using namespace cv;
9 using namespace std;
10 class FaceDetector {
11 public:
12     FaceDetector() {if (!face_cascade.load("/usr/share/OpenCV/haarcascades/
13         haarcascade_frontalface_default.xml")) {ROS_ERROR("无法加载人脸识别模型文件!");
14         ros::shutdown();

```

```

14     }
15     image_sub_ = nh_.subscribe("/usb_cam/image_raw", 1, &FaceDetector::imageCallback, this);
16     face_center_pub_ = nh_.advertise<std_msgs::Int32MultiArray>("/face_center_pixel", 10);
17     face_flag_pub_ = nh_.advertise<std_msgs::Bool>("/face_detected_flag", 10);
18     crossover_sub_ = nh_.subscribe("/px4/crossover", 10, &FaceDetector::crossoverCallback, this)
19     ;
20     namedWindow("Face Detection", WINDOW_NORMAL);
21     crossover_flag_ = 0;
22 }
23 ~FaceDetector() {destroyWindow("Face Detection");}
24 void crossoverCallback(const std_msgs::Int32::ConstPtr& msg) {crossover_flag_ = msg->data;
25 }
26 void imageCallback(const sensor_msgs::ImageConstPtr& msg) {try {
27     cout << "receive_image" << endl;
28     cout << "crossover_flag_: " << crossover_flag_ << endl;
29     Mat frame = cv_bridge::toCvShare(msg, "bgr8")->image;
30     std_msgs::Bool flag_msg;
31     flag_msg.data = false;
32     if (crossover_flag_ == 1) { Mat gray;
33         cvtColor(frame, gray, COLOR_BGR2GRAY);
34         vector<Rect> faces;
35         face_cascade.detectMultiScale(gray, faces, 1.1, 5, 0 | CASCADE_SCALE_IMAGE, Size(30,
36 30));
37         if (!faces.empty()) {
38             int center_x = faces[0].x + faces[0].width / 2;
39             int center_y = faces[0].y + faces[0].height / 2;
40             std_msgs::Int32MultiArray msg_center;
41             msg_center.data.push_back(center_x);
42             msg_center.data.push_back(center_y);
43             face_center_pub_.publish(msg_center);
44             flag_msg.data = true;
45             cout << "face_detect" << endl;
46             cout << "center_x: " << center_x << endl;
47             cout << "center_y: " << center_y << endl;
48         }
49         for (const auto& face : faces) {
50             rectangle(frame, face, Scalar(0, 255, 0), 2);
51         }
52     } else {
53         putText(frame, "Waiting for crossover signal...", Point(30, 30),
54             FONT_HERSHEY_SIMPLEX, 0.8, Scalar(0, 0, 255), 2);
55     }
56     face_flag_pub_.publish(flag_msg);
57     Mat display_frame;
58     resize(frame, display_frame, Size(320, 240), 0, 0, INTER_LINEAR);
59     imshow("Face Detection", display_frame);
60     waitKey(1);
61 } catch (cv_bridge::Exception& e) {
62     ROS_ERROR("cv_bridge异常: %s", e.what());
63 }
64 }
65 private:
66     ros::NodeHandle nh_;
67     ros::Subscriber image_sub_;
68     ros::Subscriber crossover_sub_;
69     ros::Publisher face_center_pub_;
70     ros::Publisher face_flag_pub_;
71     CascadeClassifier face_cascade;

```



```

71     int crossover_flag_;
72 };
73 int main(int argc, char** argv) {
74     ros::init(argc, argv, "face_detector");
75     FaceDetector detector;
76     ros::spin();
77     return 0;
78 }

```

Listing 4: 人脸检测代码片段

在 ROS Noetic Ubuntu 20.04 仿真验证中, Haar 级联分类器表现出色:

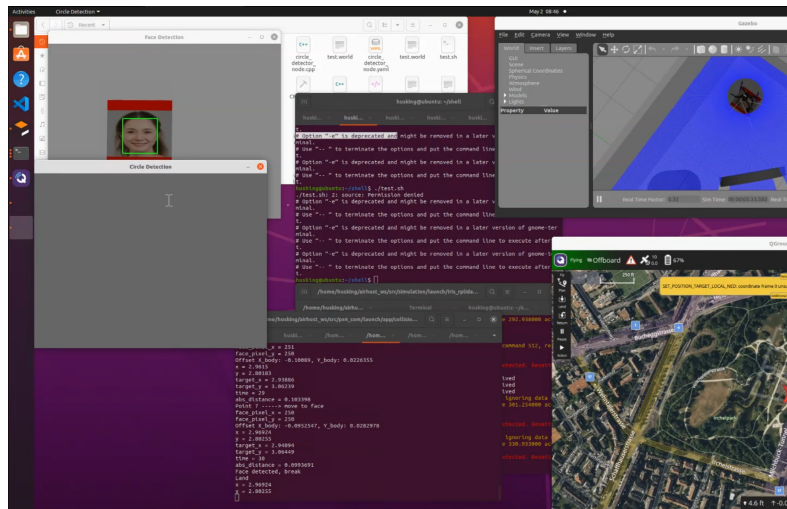


图 3: 人脸检测仿真结果

同样, 在 ROS Noetic Ubuntu 18.04 的 410 无人机实机飞行结果也令人满意:

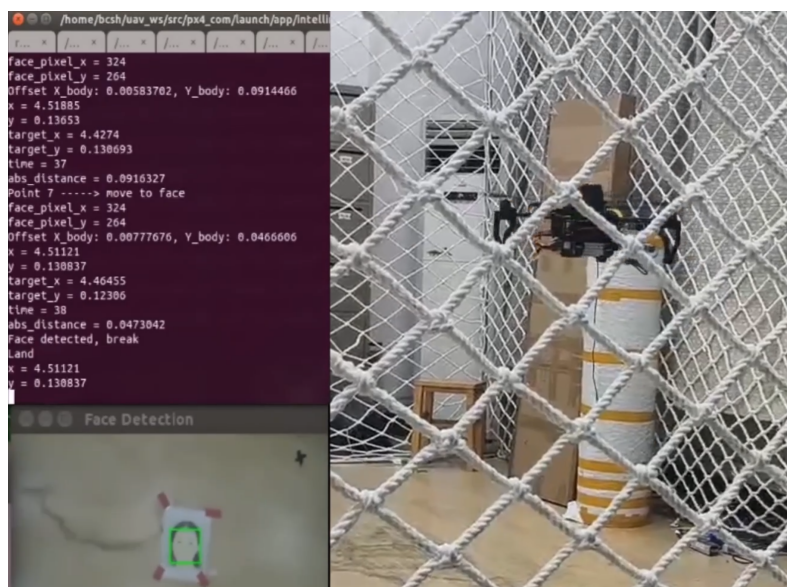


图 4: 实机飞行终端截图

降落点现场实拍:



图 5: 降落点正上方俯视图

可见无人机在人脸照片的正上方，覆盖率达到了 100%。但是在调用 YoloV5 模型的识别任务中，无人机在相同飞行高度中无法稳定识别，且识别时帧率远低于调用 Haar 级联分类器的情况。这也说明 Haar 与 Yolo 相比，在资源有限的情况下能够更好的执行任务。

6 优势、局限与发展趋势

6.1 优势

计算效率高：积分图与级联结构确保实时性；

实现简单：无须 GPU 即可部署；

资源消耗低：适合嵌入式与移动端。

6.2 局限

鲁棒性差：对光照、遮挡、姿态变化敏感；

特征表达能力有限：仅捕捉简单纹理与边缘信息；

训练成本高：需海量样本与长时间迭代。

6.3 发展趋势

深度学习方法，如 Faster R-CNN、SSD、YoloV8 等，通过端到端特征学习与更强的表征能力，已在精度与鲁棒性上超越传统方法。但在资源受限场景，如智能摄像头、无人机等，轻量级的 Haar 特征分类器或能与轻量化神经网络相结合，形成折中方案。

参考文献

- [1] VIOLA P, JONES M. Rapid Object Detection using a Boosted Cascade of Simple Features[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition: vol. 1. 2001: I-511-I-518.
- [2] DALAL N, TRIGGS B. Histograms of Oriented Gradients for Human Detection[C]//2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition: vol. 1. 2005: 886-893.
- [3] LIENHART R, MAYDT J. An Extended Set of Haar-like Features for Rapid Object Detection [C]//Proceedings of the International Conference on Image Processing: vol. 1. 2002: I-900-I-903.
- [4] YIYUAN C. Simulation on face detection based on improved adaboost algorithm[J]. Computer Simulation, 2011, 28(3): 287-290.
- [5] RONG W, PENG H, ZHEN Z. The Application of Face Detection and Tracking Method Based on OpenCV[J]. Science Technology and Engineering, 2014, 14(3): 164-167.
- [6] KE H. Design and Implementation of Fatigue Driving Detection System Based on OpenCV[D]. In Chinese. Huazhong University of Science, 2012.
- [7] TAO Z, WANZHONG C, MINGYANG L. Recognition of epilepsy electroencephalography based on AdaBoost algorithm[J]. Acta Physica Sinica, 2015, 64(12): 128701.
- [8] WEI G, YI T. Study on the cascade classifier in target detection under complex background [J]. Acta Physica Sinica, 2014, 63(4): 044207.